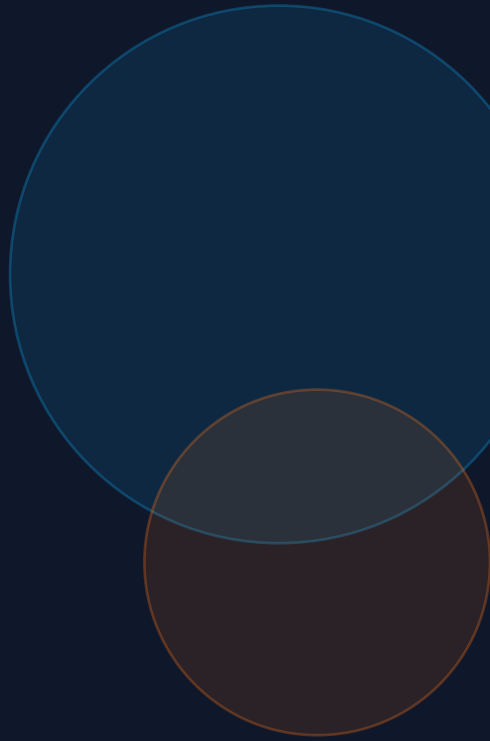
React Native

React DOM · Axios · React Navigation



Agenda

01

React Native vs React DOM

Scaffold · Similitudes · Diferencias en UI, eventos, estilos y SO

02

Comunicación con servicios REST

fetch API · Axios · Interceptors

03

React Navigation

Tipos de navegación · Props & Hooks · Paso de parámetros

React Native & React DOM

Similitudes · Diferencias · Scaffold

Scaffold — Estructura de carpetas recomendada

components/	UI pura, sin lógica (botones, inputs, tarjetas)
screens/	Cada pantalla de navegación (Home, Login…)
navigation/	Stacks, tabs y routers
services/	Comunicación con APIs REST / GraphQL
hooks/	Hooks reutilizables (useAuth, useForm…)
context/	React Contexts (login, dark mode…)
store/	Redux / Zustand — store global
utils/	Helpers: validaciones, formateo de fechas
config/	Constantes, temas, colores globales

```
my-app/
├── components/
│   └── Button.jsx
├── screens/
│   ├── Home.jsx
│   └── Login.jsx
├── navigation/
├── services/
├── hooks/
├── context/
├── utils/
└── config/
```

Similitudes — Base compartida (1/2)

Lo que aprendiste en React Web funciona igual en React Native. La lógica no cambia; cambia el entorno de ejecución.



JSX

Sintaxis de componentes



Componentes funcionales

function App() {}



Props & Estado

Flujo de datos unidireccional



Hooks

useState · useEffect · useContext



Ciclo de vida

mount · update · unmount

```
// Funciona IGUAL en Web y en React Native

import { useState, useEffect } from 'react';

export default function Ejemplo() {
  const [datos, setDatos] = useState([]);
  const [cargando, setCargando] =
    useState(true);

  useEffect(() => {
    fetch('https://api.ejemplo.com/items')
      .then(r => r.json())
      .then(data => {
        setDatos(data);
        setCargando(false);
      });
  }, []); // [] = solo al montar

  if (cargando) return <Loading />;

  return <Lista items={datos} />;
}
```

Similitudes — Patrones compartidos (2/2)

Validaciones, manejo de formularios, contextos y hooks personalizados se escriben exactamente igual. El código de negocio es 100% portable.

hooks/useFetch.js

```
// Hook 100% reutilizable Web / RN
export function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] =
    useState(true);

  useEffect(() => {
    fetch(url)
      .then(r => r.json())
      .then(setData)
      .catch(setError)
      .finally(() =>
        setLoading(false));
  }, [url]);

  return { data, error, loading };
}
```

En React Web 🖥️

```
// Web: mismo hook
const { data } = useFetch('/api/items');
return <ul>{data?.map(i =>
  <li key={i.id}>{i.name}</li>
)}</ul>;
```

En React Native 📱

```
// RN: mismo hook, distintos componentes
const { data } = useFetch('/api/items');
return (
  <FlatList data={data}
    keyExtractor={i => i.id}
    renderItem={({ item }) =>
      <Text>{item.name}</Text>}
  />
);
```

Diferencias — Renderizado de la UI

React Native no corre en un navegador. Corre directamente sobre el SO. Por eso no existe HTML ni DOM: los componentes JSX se traducen a vistas nativas reales.

🖥️ React DOM (Web)

- Se monta sobre el DOM del navegador
- Punto de entrada:
- ReactDOM.createRoot(...).render()
- JSX ⟶ elementos HTML reales
- Motor de renderizado: Blink / WebKit

📱 React Native

Web	React Native
`<div>`	`<View>`
`<p>`	`<Text>`
`<img>`	`<Image>`
`<input>`	`<TextInput>`
`<button>`	`<TouchableOpacity>`

Motor: JSI + Fabric Renderer + Hermes ⟶ próximo slide

Del JSX a la pantalla: el recorrido completo

Ejemplo: `<View style={{ padding: 16 }}> <Text>Hola Mundo</Text> </View>`

	#	Etapa	¿Qué sucede?	¿Quién participa?	Resultado
BUILD	1	**Build**	Babel traduce el JSX a llamadas de función. Metro empaqueta todo en un bundle (en release, Hermes lo precompila a bytecode).	**Babel · Metro · Hermes**	`React.createElement('View', …, React.createElement('Text', …))`
RUNTIME · en el dispositivo	2	**Ejecución JS**	Al abrir la app, Hermes ejecuta el bundle. React llama a tu componente y obtiene objetos JS que describen la interfaz.	**Hermes · React** **Hilo JS**	`React Element Tree` `{ type:'View', children:[{ type:'Text' }] }`
	3	**Shadow Tree**	React le pide a Fabric un nodo por cada elemento, llamando funciones C++ directo vía JSI. Un <Button> se descompone antes en View + Text.	**React → JSI → Fabric** **Hilo JS**	`Shadow Tree (C++)` `ViewNode → TextNode`
	4	**Layout**	Yoga recorre el Shadow Tree y aplica las reglas de Flexbox para calcular posición y tamaño de cada nodo.	**Fabric · Yoga** **Hilo de fondo**	`View  x0 y0  360×52` `Text  x16 y16`
	5	**Diff**	Fabric compara el árbol nuevo con el anterior (la primera vez, todo es nuevo) y arma la lista de cambios a aplicar.	**Fabric** **Hilo de fondo**	`Crear View · Crear Text` `Insertar Text en View`
	6	**Mount**	Las instrucciones se ejecutan en el hilo UI de Android: se instancian las vistas nativas reales.	**Fabric (capa Android)** **Hilo UI**	`ReactViewGroup` `ReactTextView`
	7	**Dibujo**	Android hace su ciclo habitual measure → layout → draw.	**Sistema Android** **Hilo UI**	`Píxeles en pantalla`

¿Y con un setState? Se repiten los pasos 2 a 6 y el diff produce un único cambio: actualizar el texto.

¿Qué hay en el APK cuando hago el development build? El motor nativo (Hermes, Fabric, Yoga) + tu JS. Tu UI no se traduce a código nativo: se construye en runtime.

Fuente: https://reactnative.dev/architecture/render-pipeline

Diferencias — Manejo de Eventos

En la web hay mouse, teclado y scroll de navegador. En móvil hay dedo. El sistema de eventos de React Native refleja eso.

🖥️ React DOM (Web)

- onClick
- onChange
- onSubmit
- onMouseOver / onMouseEnter
- onFocus / onKeyDown

📱 React Native

- onPress (≈ click — cualquier tap)
- onChangeText (devuelve string directo)
- Se maneja en el botón con onPress
- ❌ No existe — no hay mouse
- onFocus / onBlur en TextInput

Diferencias — Estilos

En la web Flexbox es una opción. En React Native es el único sistema de layout. Y la propiedad CSS se convierte a camelCase: background-color ⟶ backgroundColor.

🖥️ React DOM (Web)

- CSS externo (.css, className)
- Bootstrap / Tailwind / SCSS
- Selectores: .clase, #id, :hover
- Media queries nativas
- Float, Grid, Flexbox, position

📱 React Native

- Sin CSS externo, sin className
- Objetos JS: style={{ ... }}
- StyleSheet.create({ ... })
- Solo Flexbox (direction: column)
- Dimensions / libs para responsive

Conversión de propiedades CSS ⟶ JavaScript (camelCase)

background-color	font-size	border-radius	padding-top	z-index
↓	↓	↓	↓	↓
backgroundColor	**fontSize**	**borderRadius**	**paddingTop**	**zIndex**

Diferencias — Acceso al Sistema Operativo

En la web el entorno es el navegador. En React Native el entorno es el sistema operativo real: accedés a la cámara, GPS, notificaciones y almacenamiento nativo.

🖥️ React DOM (Web)

- `localStorage / sessionStorage`
- `window.innerHeight / innerWidth`
- `navigator.geolocation.getCurrentPosition()`
- `addEventListener('scroll')`
- `document.querySelector`

📱 React Native

- `AsyncStorage / SecureStore`
- `Dimensions.get('window').width`
- `expo-location / @react-native-community/geolocation`
- `expo-camera / react-native-camera`
- `expo-notifications / FCM / OneSignal`

Comunicación con APIs REST

fetch · Axios · Interceptors

fetch API — Peticiones HTTP nativas

fetch es una API nativa de JavaScript para realizar peticiones HTTP. Devuelve una promesa y es soportada en React Native sin instalación adicional.

GET

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error('Error en la petición');
    }
    return response.json();
  })
  .then(data => {
    console.log('Datos:', data);
  })
  .catch(error => {
    console.error('Error:', error);
  });
```

POST

```
fetch('https://api.example.com/submit', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    name: 'John', age: 30
  })
})
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Axios — Librería HTTP con superpoderes

```
npm install axios
```

- ✓ Async / Await nativo
- ✓ Manejo automático JSON
- ✓ Interceptors de request/response
- ✓ Interfaz más simple que fetch

GET

```
axios.get('https://api.example.com/data')
  .then(response => {
    // response.data ya es JSON
    console.log(response.data);
  })
  .catch(error => {
    console.error('Error:', error);
  });
```

Interceptors

```
axios.interceptors.request.use(
  config => {
    config.headers.Authorization =
      'Bearer ' + token;
    return config;
  },
  error => Promise.reject(error)
);
```

React Navigation

Stack · Tabs · Drawer · Hooks & Props

¿Qué es React Navigation?

React Navigation es la librería oficial y recomendada para manejar la navegación entre pantallas en aplicaciones React Native. React Native no incluye navegación por defecto, a diferencia de la web donde existen rutas como /home, /about.

Stack Navigator



Pilas de pantallas
navegar adelante / atrás

Tab Navigator



Navegación con pestañas
como Instagram

Drawer Navigator



Menú deslizable
como Gmail

React Navigation — Componentes obligatorios

Para que React Navigation funcione se necesitan exactamente tres piezas, siempre en este orden jerárquico.

Componente	Dónde va	Qué hace
`NavigationContainer`	App.js (raíz)	Contexto global que gestiona el estado de navegación
`<X>.Navigator`	navigation/AppNavigator.jsx	Define el tipo de navegación (Stack, Tab, Drawer)
`<X>.Screen`	Dentro del Navigator	Registra cada pantalla con su nombre y componente

⚠️ **Sin NavigationContainer...**

React Navigation no tiene contexto y cualquier intento de navegar lanza: Couldn't find a navigation object. Siempre debe ser el componente raíz que envuelve toda la app.

```
// Jerarquía obligatoria
<NavigationContainer>
        <Stack.Navigator>
                <Stack.Screen />  <Stack.Screen />  ...
```

Es equivalente al BrowserRouter de React Router Web.

NavigationContainer y AppNavigator — Código completo

App.js

```
import { NavigationContainer }
  from '@react-navigation/native';
import AppNavigator
  from './navigation/AppNavigator';

export default function App() {
  return (
    <NavigationContainer>
      <AppNavigator />
    </NavigationContainer>
  );
}
```

navigation/AppNavigator.jsx

```
import { createNativeStackNavigator }
  from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

export default function AppNavigator() {
  return (
    <Stack.Navigator
      initialRouteName="Home"
      screenOptions={{
        headerStyle: {
          backgroundColor: colors.primary },
        headerTintColor: colors.white,
```

```
// Pantallas registradas en el Navigator

      <Stack.Screen name="Home"         component={HomeScreen}    options={{ title: 'PokeExplorer' }} />
      <Stack.Screen name="PokemonList"   component={PokemonListScreen}  options={{ title: 'Pokedex' }} />
      <Stack.Screen name="PokemonDetail" component={PokemonDetailScreen}
        options={({ route }) => ({ title: route.params.name })} />
    </Stack.Navigator>
  );
}
```

initialRouteName

Pantalla al abrir la app. Si se omite, usa la primera Screen.

screenOptions

Opciones compartidas por todas las pantallas.

options por Screen

Sobreescribe screenOptions; acepta función con { route }.

Paso de parámetros entre pantallas

Para enviar datos de una pantalla a otra se usa el segundo argumento de navigate. route.params es un objeto plano: podés pasar strings, números u objetos serializables.

📤 Quien ENVÍA — PokemonListScreen.jsx

```
export default function PokemonListScreen(
  { navigation }
) {
  return (
    <FlatList
      data={pokemons}
      renderItem={({ item }) => (
        <TouchableOpacity
          onPress={() =>
            navigation.navigate(
              'PokemonDetail',
              { name: item.name }  // <-- envía
            )
          }
        >
          <Text>{item.name}</Text>
        </TouchableOpacity>
      )}
    />
  );
}
```

{ name: 'pikachu' }

📥 Quien RECIBE — PokemonDetailScreen.jsx

```
export default function PokemonDetailScreen(
  { route, navigation }
) {
  // Lee el parámetro enviado
  const { name } = route.params;

  return (
    <View>
      <Text>
        Detalle de: {name}
      </Text>
      <Button
        title="Volver"
        onPress={() =>
          navigation.goBack()}
      />
    </View>
  );
}
```

El prop navigation

React Navigation inyecta automáticamente el prop navigation a todos los componentes registrados como pantalla dentro de un Navigator (ej. Stack.Screen).

`navigation.navigate('Pantalla')`	*Ir a una pantalla*
`navigation.goBack()`	*Volver atrás*
`navigation.replace('Otra')`	*Reemplazar la pantalla actual*
`navigation.navigate('P', { id: 5 })`	*Navegar pasando parámetros*

```
function HomeScreen({ navigation }) {
  return <Button title="Ir a detalles" onPress={() => navigation.navigate('Details')} />;
}
```

Hooks de navegación — useNavigation y useRoute

useNavigation()

Para componentes que no son pantalla pero necesitan navegar

```
import { useNavigation } from
  '@react-navigation/native';

function MiBoton() {
  const navigation = useNavigation();
  return (
    <Button title="Volver a Home"
      onPress={() =>
        navigation.navigate('Home')}
    />
  );
}
```

useRoute()

Para leer parámetros pasados desde cualquier componente

```
import { useRoute } from
  '@react-navigation/native';

// Quien navega:
navigation.navigate('Detalles',
  { nombre: 'Juan', edad: 25 });

// DetallesScreen.js:
function DetallesScreen() {
  const route = useRoute();
  const { nombre, edad } =
    route.params;
}
```

Resumen — Hooks & Props de React Navigation

Hook / Prop	¿Para qué sirve?	¿Dónde se usa?
navigation prop	Navegar entre pantallas	En pantallas (Stack.Screen)
route prop	Leer parámetros pasados	En pantallas
useNavigation()	Navegar desde componentes hijos	En cualquier componente
useRoute()	Leer parámetros desde cualquier comp	En cualquier componente
useFocusEffect()	Ejecutar efecto al enfocar pantalla	En pantallas (avanzado)
useIsFocused()	Saber si la pantalla está visible	En pantallas o hooks